



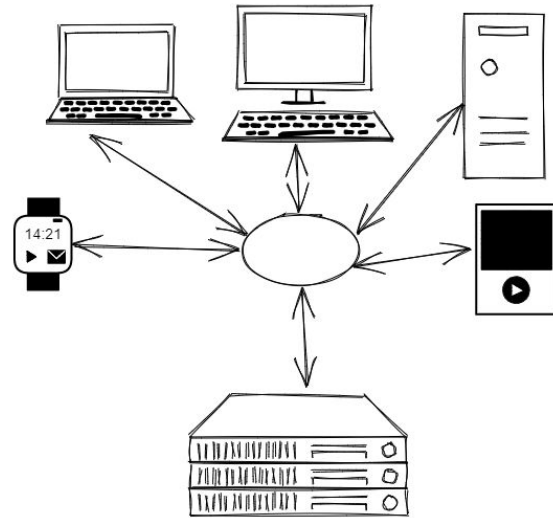
APIs alternativas 1

Comunicación Cliente Servidor

Como ya hablamos la comunicación básica en un sistema se da entre dos nodos generando una comunicación “cliente servidor”.

Estos roles tradicionalmente tomados por “computadoras” ahora son tomados por diversos dispositivos:

- » Dispositivos móviles.
- » Sensores.
- » IoT.
- » Electrodomésticos.
- » Maquinarias.



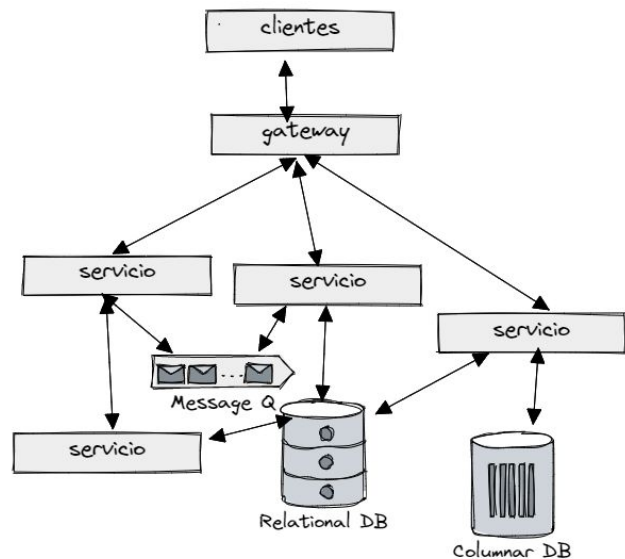
Comunicación Cliente Servidor

Y los “servicios” se van complejizando de manera que los mismos se resuelven a través de la colaboración de diversos componentes, que se comunican entre ellos.

Esto provoca que existan diversas formas de establecer comunicación entre dos puntos de un sistema.

gRPC nos sirvió como ejemplo para analizar internamente una manera de comunicación.

Hoy veremos algunos de los estándares alternativos más populares para generar APIs de consumo de servicios.





APIs

La forma de la API es el punto central a definir ya que es la que indica la semántica y las operaciones posibles a la hora de consumir un servicio.

Para implementarlas existen diversos paradigmas, estándares, frameworks, y tecnologías a elegir.





APIs

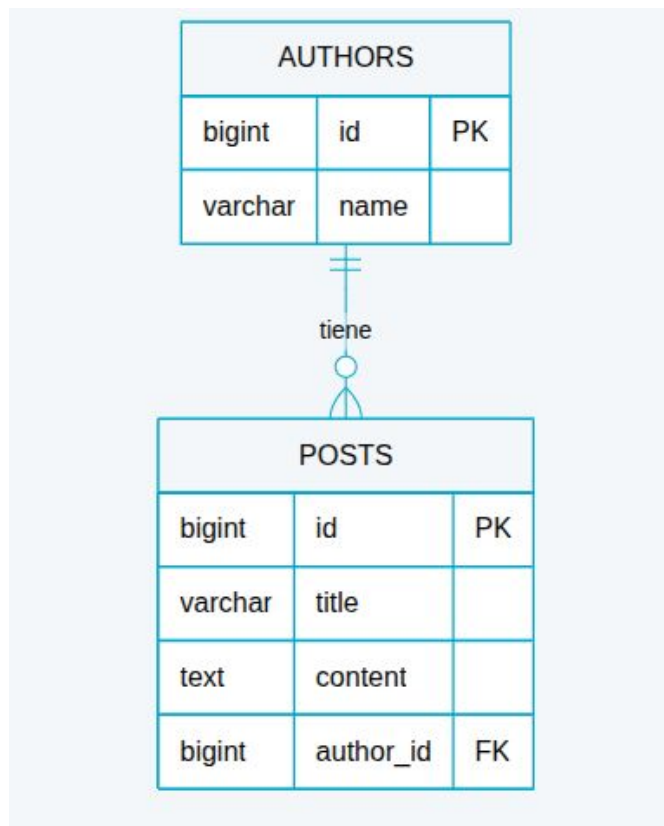
A pesar de que cualquiera de ellas puede utilizarse para resolver casi cualquier API se pueden tomar diversos criterios para elegir la “mejor” para el servicio que estamos implementando.

- Formas de comunicación y periodicidad.
 - Transporte.
 - Diversidad de clientes, lenguajes, tecnologías.
 - Maneras de describir las operaciones.
 - Diversidad de los payloads.
- 

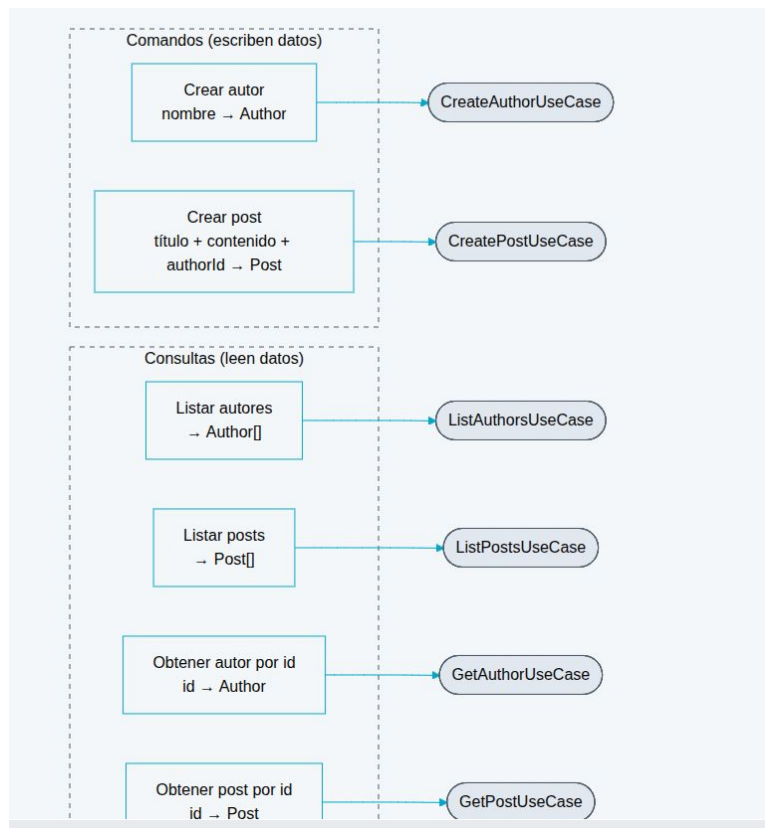


Servicio

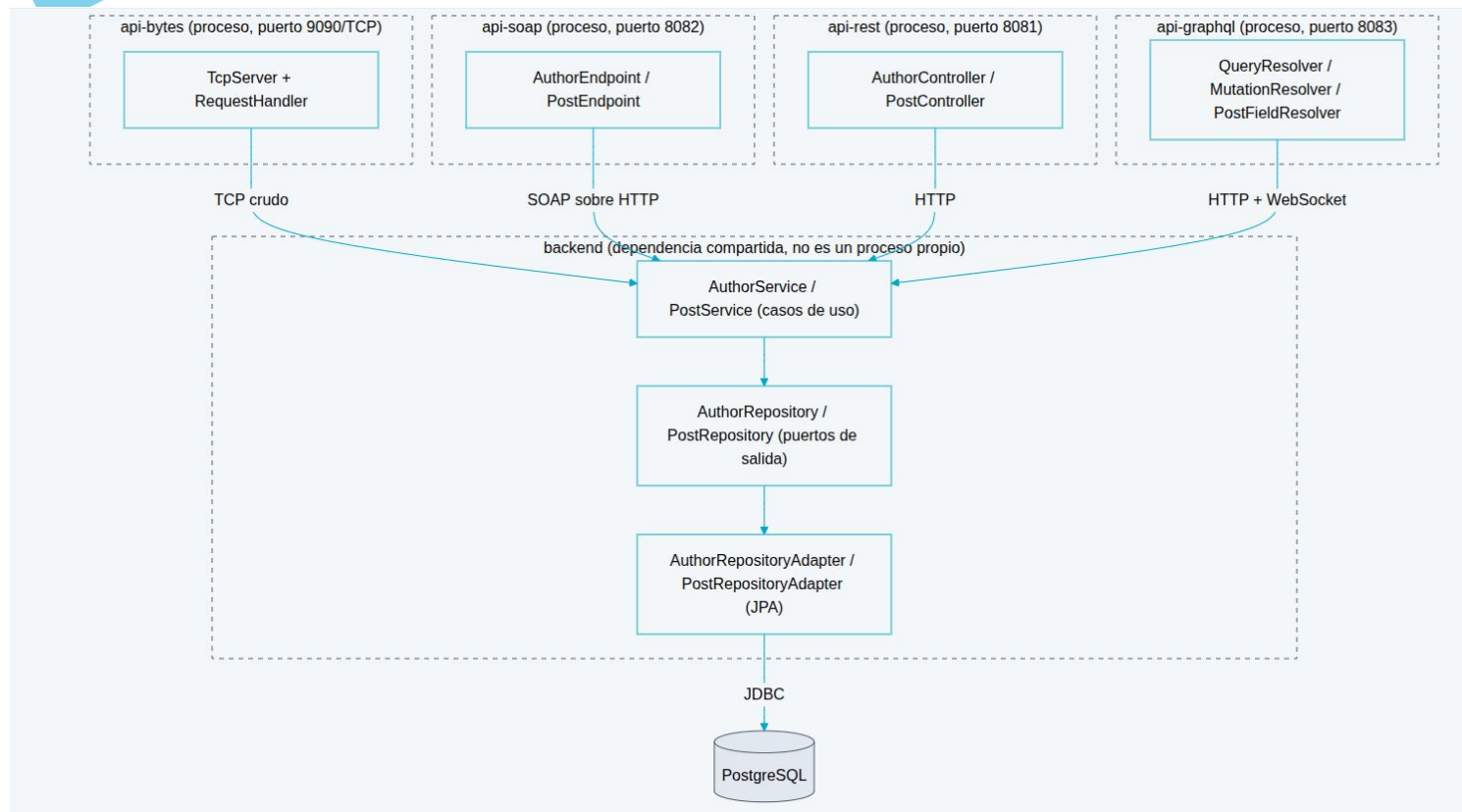
Servicio de Ejemplo



Servicio de Ejemplo



Servicio de Ejemplo



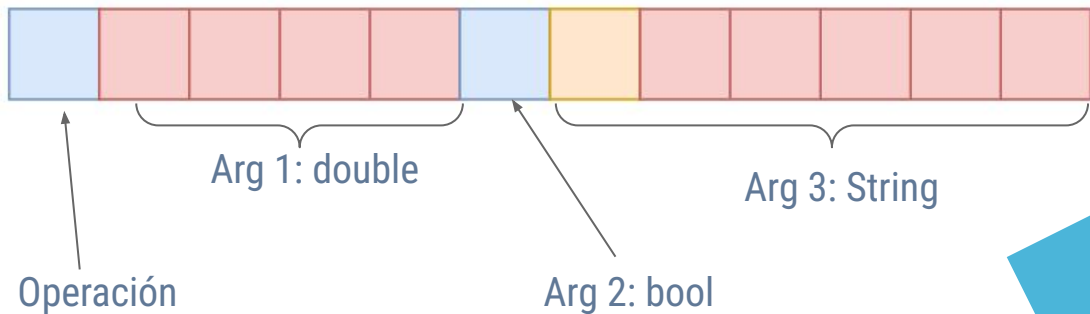


Tira de Bytes

Tira de Bytes

La tira de Bytes es la forma más “primitiva” y/o “básica” de generar una API.

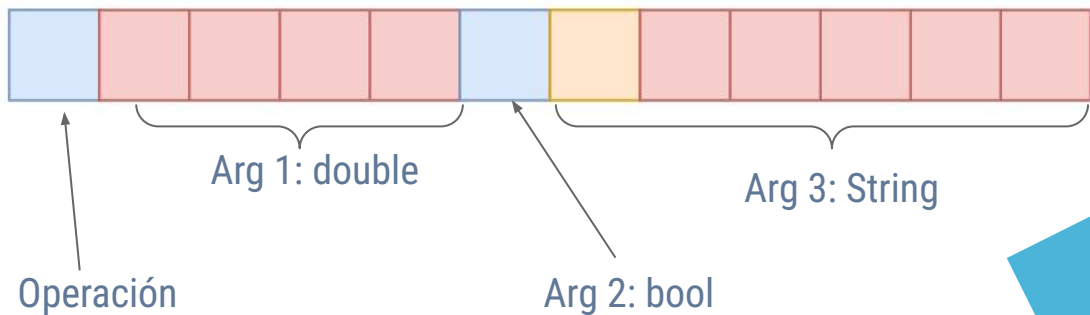
Durante mucho tiempo fue la única opción hasta que se generaron nuevas y mejores alternativas.



Tira de Bytes

Pero al día de hoy todavía pueden encontrarse algunas API legacy que usan alguna variante de la misma.

Se trata de abrir una conexión por socket y convenir (mediante el uso de documentación) de que manera se envía los diferentes valores del request y la response.





Tira de Bytes

La documentación del Request

Byte(s)	Camo
1	opcode (ver tabla de abajo)
...	campos del opcode, en orden



Tira de Bytes

Documentación de cada operación en el Request:

Opcode	Valor	Campos de request
CREATE_AUTHOR	0x01	UTF name
CREATE_POST	0x02	long authorId, UTF title, UTF content
LIST_AUTHORS	0x03	(sin campos)
LIST_POSTS	0x04	boolean filterByAuthor; si es true, además long authorId

Tira de Bytes

La documentación del Response:

Byte(s)	Campo
1	status: 0x00 OK, 0x01 ERROR
...	si ERROR: UTF message. Si OK: payload según el opcode



Tira de Bytes

Documentación de cada operación en el Request:

Opcode

Payload de response (con status OK)

CREATE_AUTHOR long id, UTF name

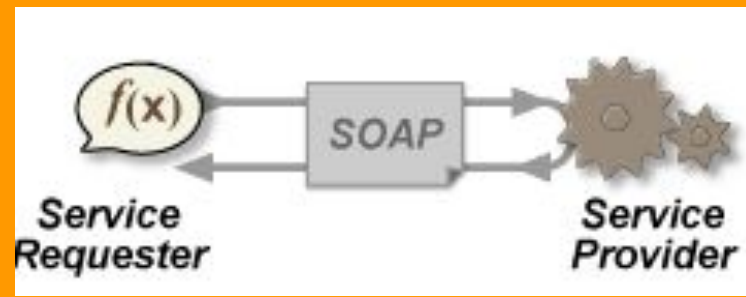
CREATE_POST long id, UTF title, UTF content, long authorId

LIST_AUTHORS int count, seguido de count autores (long id, UTF name)

LIST_POSTS int count, seguido de count posts (long id, UTF title, UTF content, long authorId)



SOAP





SOAP

SOAP (Simple Object Access Protocol) es un protocolo estándar de comunicación mediante el intercambio de datos utilizando XML.





SOAP - Características

La definición de los servicios también se hace mediante XML.





SOAP - Características

El protocolo tiene 3 características principales:

- **Extensibilidad:** El protocolo pueden extenderse para hacer servicios complejos o agregar seguridad y routing
 - **Neutralidad:** Utiliza transporte por TCP y puede aplicarse diversos protocolos de aplicación como HTTP, SMTP o JMS.
 - **Independencia:** (permite cualquier modelo de programación) al definirse vía XML se puede programar en diversos lenguajes y paradigmas.
- 



SOAP - Mensaje

Un mensaje en SOAP consta de las siguientes partes:

- **Envelope:** Elemento que engloba
 - **Header:** Elemento opcional que tiene atributos que son complementarios al mensaje, por ejemplo, elementos de autenticación.
 - **Body:** Elemento obligatorio con el contenido del mensaje a enviar
 - **Fault:** Información de errores que se produjeron (si se produjeron).
- 

SOAP - Mensaje

Ejemplo de la estructura de un mensaje.

```
<?xml version="1.0"?>

<SOAP-ENV:Envelope xmlns:SOAP-ENV = "http://www.w3.org/2001/12/soap-envelope"
    SOAP-ENV:encodingStyle = "http://www.w3.org/2001/12/soap-encoding">

    <SOAP-ENV:Header>

    </SOAP-ENV:Header>

    <SOAP-ENV:Body>

        <SOAP-ENV:Fault>

        </SOAP-ENV:Fault>

    </SOAP-ENV:Body>

</SOAP_ENV:Envelope>
```

SOAP - Ejemplo mensajes

Ejemplo de un mensaje en un request en xml.

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"  
  xmlns:cat="http://itba.edu.ar/pod/api/soap">  
  <soapenv:Header/>  
  <soapenv:Body>  
    <cat:createAuthorRequest>  
      <cat:name>Julio Cortázar</cat:name>  
    </cat:createAuthorRequest>  
  </soapenv:Body>  
</soapenv:Envelope>
```

SOAP - Ejemplo mensajes

Ejemplo de un mensaje un response en xml.

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
  <soapenv:Body>
    <cat:createAuthorResponse xmlns:cat="http://itba.edu.ar/pod/api/soap">
      <cat:author>
        <cat:id>1</cat:id>
        <cat:name>Julio Cortázar</cat:name>
      </cat:author>
    </cat:createAuthorResponse>
  </soapenv:Body>
</soapenv:Envelope>
```

SOAP - Ejemplo mensajes

Ejemplo de un mensaje de error un response en xml.

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
  <soapenv:Body>
    <soapenv:Fault>
      <faultcode>soapenv:Client</faultcode>
      <faultstring>Author not found: 999</faultstring>
    </soapenv:Fault>
  </soapenv:Body>
</soapenv:Envelope>
```



SOAP - Definición de un servicio

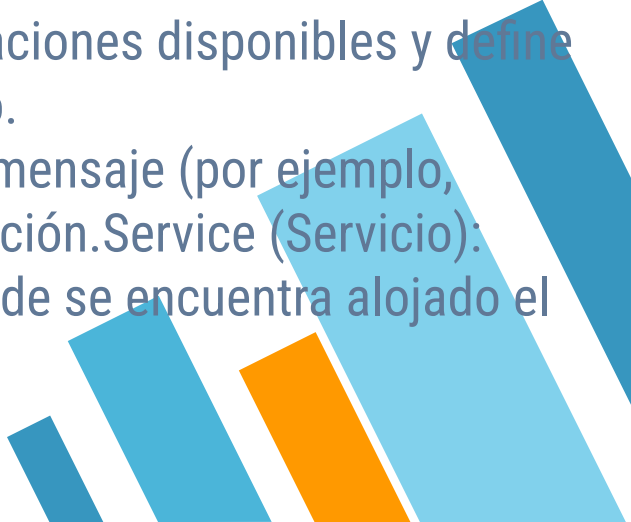
Un servicio en SOAP se define mediante un archivo de *contrato* en formato XML llamado **WSDL (Web Services Description Language)**, el cual especifica las operaciones disponibles, los tipos de datos y la ubicación del servidor





SOAP - Definición de un servicio

Un archivo WSDL utiliza varias etiquetas principales para estructurar el servicio:

- » **Types (Tipos):** Define los tipos de datos y estructuras XML personalizados que se van a usar (normalmente mediante esquemas XSD).
 - » **Message (Mensajes):** Representa los datos que se intercambian, separando los parámetros de entrada y salida de cada operación.
 - » **PortType (Interfaz/Operaciones):** Agrupa las operaciones disponibles y define qué mensajes de entrada y salida usa cada método.
 - » **Binding (Enlace):** Especifica cómo se transmite el mensaje (por ejemplo, usando SOAP sobre HTTP) y el formato de codificación.
 - » **Service (Servicio):** Define la dirección web física (endpoint o URL) donde se encuentra alojado el servicio listo para ser invocado.
- 



SOAP - Definición de un servicio

Un servicio en SOAP se define mediante un archivo de contrato en formato XML llamado **WSDL (Web Services Description Language)**, el cual especifica las operaciones disponibles, los tipos de datos y la ubicación del servidor



SOAP - Definición de un servicio

Aunque se puede tener todo junto hay partes que se pueden separar en un archivo **xsd**.

Característica	WSDL (<i>Web Services Description Language</i>)	XSD (<i>XML Schema Definition</i>)
Propósito principal	Describir el servicio web completo (operaciones, rutas, protocolos).	Describir el contenido del mensaje (campos, tipos de datos, restricciones).
Nivel de abstracción	Macro: Define la interfaz de comunicación y los puntos de acceso (endpoints).	Micro: Define el esqueleto y las reglas de los fragmentos XML individuales.
Pregunta que responde	<i>¿A qué URL envío la petición y qué funciones puedo llamar?</i>	<i>¿Qué campos lleva el XML de la petición y de qué tipo son?</i>
Componentes clave	<code><portType></code> , <code><binding></code> , <code><service></code> , <code><operation></code> .	<code><element></code> , <code><complexType></code> , <code><simpleType></code> , <code><restriction></code> .



SOAP - Definición de un servicio

El xsd tiene la definición de tipos complejos reutilizables

```
<xs:complexType name="author">
  <xs:sequence>
    <xs:element name="id" type="xs:long"/>
    <xs:element name="name" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
```



SOAP - Definición de un servicio

y los elementos de request/response

```
<xsd:element name="createAuthorRequest">
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element name="name" type="xsd:string"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
```

SOAP - Definición de un servicio

message: Por cada elemento del XSD, Spring-WS declara un wsdl:message que simplemente lo envuelve como una única part:

```
<wsdl:message name="createAuthorRequest">
  <wsdl:part element="tns:createAuthorRequest" name="createAuthorRequest"/>
</wsdl:message>
<wsdl:message name="createAuthorResponse">
  <wsdl:part element="tns:createAuthorResponse" name="createAuthorResponse"/>
</wsdl:message>
```

SOAP - Definición de un servicio

portType: Es la interfaz abstracta del servicio: qué operaciones existen y qué mensajes entran/salen de cada una

```
<wsdl:portType name="CatalogPort">
  <wsdl:operation name="createAuthor">
    <wsdl:input message="tns:createAuthorRequest"/>
    <wsdl:output message="tns:createAuthorResponse"/>
  </wsdl:operation>
  <wsdl:operation name="createPost">...</wsdl:operation>
  <wsdl:operation name="listAuthors">...</wsdl:operation>
  <wsdl:operation name="listPosts">...</wsdl:operation>
</wsdl:portType>
```

SOAP - Definición de un servicio

binding: Dice cómo viajan esos mensajes por la red: qué protocolo de transporte y qué estilo de serialización

```
<wsdl:binding name="CatalogPortSoap11" type="tns:CatalogPort">
  <soap:binding transport="http://schemas.xmlsoap.org/soap/http" style="document"/>
  <wsdl:operation name="createAuthor">
    <soap:operation soapAction=""/>
    <wsdl:input><soap:body use="literal"/></wsdl:input>
    <wsdl:output><soap:body use="literal"/></wsdl:output>
  </wsdl:operation>
</wsdl:binding>
```

SOAP - Definición de un servicio

service / port: El endpoint concreto y accesible en la red: une el portType abstracto con el binding y le da una URL física.

```
<wsdl:service name="CatalogPortService">
  <wsdl:port name="CatalogPortSoap11" binding="tns:CatalogPortSoap11">
    <soap:address location="http://localhost:8082/ws"/>
  </wsdl:port>
</wsdl:service>
```



SOAP - Generar código

Los lenguajes proveen de compiladores que permiten interpretar las definiciones del xml y generar las clases de servicios y stubs para que luego se pueda completar la implementación del servicio.

O generar los xmls a partir del código.





SOAP - Generar código

El proceso es similar al visto con gRPC, varían las clases a implementar y los nombres.

En Java existen librerías como:

- » Spring WS
- » CXF
- » AXIs
- » soporte nativo vía JAX-WS.

La necesidad de usar uno u otro puede estar dada por el código existente.



SOAP - Código

Definición del servicio en código.

```
@PayloadRoot(namespace = NAMESPACE, localPart = "createPostRequest")
@ResponsePayload
public CreatePostResponse createPost(@RequestPayload CreatePostRequest request) {
    try {
        var post = createPostUseCase.createPost(
            new CreatePostCommand(request.getAuthorId(),
                                request.getTitle(), request.getContent()));
        CreatePostResponse response = new CreatePostResponse();
        response.setPost(toXml(post));
        return response;
    } catch (AuthorNotFoundException ex) {
        throw new AuthorNotFoundSoapFault(ex.getMessage());
    }
}
```

SOAP - Código

Uso del DTO;

```
private static Post toXml(ar.edu.itba.pod.api.backend.domain.Post post) {  
    Post xml = new Post();  
    xml.setId(post.id());  
    xml.setTitle(post.title());  
    xml.setContent(post.content());  
    xml.setAuthorId(post.authorId());  
    return xml;  
}
```



SOAP - Ventajas e inconvenientes

Durante mucho tiempo SOAP fue el sistema de web services más popular debido a sus características de extensibilidad y multi lenguaje.

Las contra que tiene son que la definición via xml, puede ser muy “verbosa”, que la definición de servicios puede ser compleja (aún para servicios simples).





SOAP - Ventajas e inconvenientes

Entonces fue perdiendo popularidad contra REST y se generó una dicotomía donde
REST se usa para servicios “simples” web y B2C;
y SOAP se usa para aquellos más complejos con necesidades de mayor seguridad como Financieras y B2B





Rest



Rest

Según la definición original escrita en la tesis doctoral de Roy T. Fielding REST (representational state transfer) es:

“Estilo arquitectónico para sistemas distribuidos de hipermedia, describiendo los principios de ingeniería de software que guían REST y las constantes de interacción elegidas para retener esos principios”

Entonces originalmente era un conjunto de principios de arquitectura. Como Fielding trabajó en el protocolo HTTP y los principios de REST están muy atados a ese protocolo el REST tomó un significado más amplio.





Rest

Entonces en estos tiempos se podría describir a Rest no oficialmente como

Una interfaz remota de servicios basada en **Recursos** sobre el protocolo HTTP. Esta interfaz se utiliza para obtener y modificar datos multimedia y texto en diversos formatos como JSON, XML, binario, etc.

Rest probablemente sea la interfaz web más popular por esa conexión con HTTP.

La principal diferencia con otros tipos de interfaces como RPC es la definición de los servicios a través de **Recursos** (sustantivos) en vez de **Verbos** (acciones).





Rest - Principios de Arquitectura

1. **Arquitectura cliente-servidor (Client-Server):** Para lograr separación de responsabilidades y desacoplar la interacción entre el cliente y el servidor. Como la dependencia es a través del mensaje modificar cada componente no debería afectar al otro.
- 



Rest - Principios de Arquitectura

2. **Ausencia de estado (Stateless):** El estado/contexto de la petición es mantenida por el cliente. El servidor es stateless y recibe todo del cliente. Esto va a permitir que diversas instancias del servicio puedan atender llamadas sucesivas de un mismo cliente.
- 



Rest - Principios de Arquitectura

3. **Habilitación y uso del cache (Cacheable):** Todos los recursos deben ser cacheables, mientras y por el tiempo que tenga sentido. Esto permite optimizar la comunicación entre el cliente y el servidor
- 



Rest - Principios de Arquitectura

4. **Sistema por capas (Layered):** Un cliente debe conocer únicamente la capa a la que le está hablando. Es decir, no debe tener en cuenta aspectos concretos





Rest - Principios de Arquitectura

5. **Ejecución de código (Code on Demand):** Esto era opcional y con el tiempo entró en desuso. Era la posibilidad de poder ejecutar código en el cliente (o desde el cliente) mediante el uso de scripts o applets
- 



Rest - Principios de Arquitectura

6. **Interfaz uniforme:** se aplica el principio de “generalidad” para simplificar las interacciones. Para ello define las siguientes restricciones:
 - a. **Identificación de recursos en las peticiones:** Todos los recursos se identifican unívocamente. Esto es independiente de cual es la representación que se envía (que se puede solicitar) por medio de headers.
 - b. **Manipulación de recursos a través de representaciones:** Cuando el cliente obtiene la información de un recurso la misma debe contener toda la información para poder modificar/borrar a dicho recurso.
 - c. **Mensajes auto-descriptivos:** cada mensaje debe contener toda la información necesaria para que se ejecute.
 - d. **Hipermedia como motor del estado de la aplicación:** a partir de un pedido inicial el cliente debería poder “descubrir” todos los servicios del sistema
- 



Rest - Recursos

Los recursos son la información que diferentes aplicaciones proporcionan a sus clientes. Los recursos pueden ser imágenes, videos, texto, números o cualquier tipo de datos.





Rest - Recursos

Los recursos pueden ser de 3 tipos:

- **colección:** un conjunto de elementos que tienen características similares. Se identifican con el sustantivo que la describe (generalmente plural).
 - **singleton/documento:** una instancia individual de una colección. Se identifica mediante un identificador natural o interno y contiene el resto de los datos que contienen.
 - **subcolecciones:** son conjuntos de singletons que están subordinados a un elemento.
- 



Rest - Recursos - URIs

Para acceder a los recursos se utilizan URIs.

Partiendo de la uri del servicio y potencialmente un contexto y versionado, por ejemplo: <http://remote-service/context/v1>

Jerárquicamente se agrega componentes

- **colecciones:** con el nombre del recurso: `/movies/`
 - **singleton:** con la colección y el identificador: `/movies/{id}`
 - **subcolección:** agregamos la colección: `/movies/{id}/actors`
- 



Rest - Recursos - URIs

Se utilizan las features de HTTP para hacer más específicos el pedido:

- **status codes:** para indicar resultados.
 - **content-types:** para pedir/indicar la representación a usar en el recurso
 - **query params:** para agregar limitantes como criterios de búsqueda, paginación, etc.
- 

Rest - Recursos - Ejemplo

Documentación de urls, status, y métodos

Método	URL	Status OK	Errores
POST	/authors	201 Created + Location	400 (validación)
GET	/authors	200	—
GET	/authors/{id}	200	404 si no existe
GET	/authors/{id}/posts	200	404 si el autor no existe
POST	/posts	201 Created + Location	400 (validación), 404 si authorId no existe
GET	/posts	200	—
GET	/posts/{id}	200	404 si no existe



Rest - Recursos - Content types

```
curl -H "Accept: application/json" http://localhost:8081/authors/1  
curl -H "Accept: application/hal+json" http://localhost:8081/authors/1  
curl -H "Accept: application/xml" http://localhost:8081/authors/1
```



Rest - Recursos - HATEOAS

Hipermedia como motor del estado de la aplicación es una de las restricciones de REST.

Básicamente indica que cada cliente debería poder navegar el servicio mediante la respuesta al primer Request. La manera de navegar es agregar a la respuesta hipermedia links a los recursos dependientes (subcolecciones o singletons).

```
{
  "id": 1,
  "name": "Julio Cortázar",
  "_links": {
    "self": { "href": "http://localhost:8081/authors/1" },
    "posts": { "href": "http://localhost:8081/authors/1/posts" }
  }
}
```



Rest - Código

La mayoría de los lenguajes tienen frameworks que funcionan como middleware para poder crear aplicaciones REST simples.

Dentro del mundo Java tenemos varios, los ejemplos presentados a continuación combinan elementos de Spring - Jersey - JAX-RS para poder proveer un servicio que retorna un recurso llamado message representado en JSON



Rest - Código - Objeto

Definición del objeto que representa al Recurso, en este caso un autor

```
import org.springframework.hateoas.RepresentationModel;

public class AuthorModel extends RepresentationModel<AuthorModel> {

    private final Long id;
    private final String name;

    public AuthorModel(Long id, String name) {
        this.id = id;
        this.name = name;
    }
}
```

Rest - Código - Controlador

```
@RestController
@RequestMapping("/authors")
public class AuthorController {

    private static final String[] REPRESENTATIONS = {
        MediaType.APPLICATION_JSON_VALUE,
        MediaTypes.HAL_JSON_VALUE,
        MediaType.APPLICATION_XML_VALUE
    };

    @GetMapping(produces = {MediaType.APPLICATION_JSON_VALUE,
        MediaTypes.HAL_JSON_VALUE,
        MediaType.APPLICATION_XML_VALUE})
    public CollectionModel<AuthorModel> listAuthors() {
        var models = listAuthorsUseCase.listAuthors().stream()
            .map(authorAssembler::toModel).toList();
        return CollectionModel.of(models, linkTo(methodOn(AuthorController.class)
            .listAuthors()).withSelfRel());
    }
}
```



Rest - Ventajas

REST al ser uno de los estándares más populares tiene un gran soporte en lenguajes, frameworks y librerías. Además de conocimiento colectivo entre la comunidad de desarrollo:

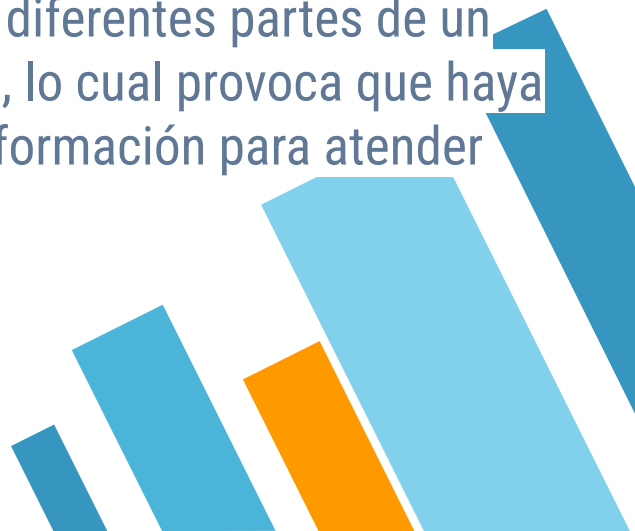
Por las restricciones que lo definen está armado para que sea:

- **escalable:** stateless, client-server, cacheable
 - **flexible:** client-server y las formas de las representaciones
 - **independiente de la tecnología:** como el transporte es http tanto cliente y servidor pueden estar escrito en lenguajes diferentes
- 



Rest - Desventajas

En contrapartida algunas desventajas de REST puede ser

- Al no haber un estándar definido surgen inconvenientes cuando se quieren describir sistemas o partes que no son fácilmente describibles a través de sustantivos (y/o operaciones CRUD).
 - En ciertas situaciones se vuelve incómodo cuando diferentes partes de un sistema requieren diferentes facetas de un recurso, lo cual provoca que haya varios endpoints o uno que devuelva demasiada información para atender todos los casos.
- 



CREDITS

Content of the slides:

- » POD - ITBA

Images:

- » POD - ITBA
- » Or obtained from: commons.wikimedia.org

Slides theme credit:

Special thanks to all the people who made and released these awesome resources for free:

- » Presentation template by [SlidesCarnival](https://slidescarnival.com)
 - » Photographs by [Unsplash](https://unsplash.com)
- 